quick question on trading intraday options ... as NSE PROVIDES OI DATA ONLY at 3-5 minute interval

3:25 pm

While the official NSE India Option Chain updates its Open Interest (OI) data in 3-to-5-minute intervals, intraday options traders and scalpers bypass this delay by using **broker-integrated, real-time analytics platforms** that offer tick-by-tick OI updates. Sensibull +4

How to Trade Intraday Options Faster

- **Use Analytical Platforms:** Instead of relying solely on the exchange's delayed updates, actively trade via broker platforms that stream real-time options data, like Sensibull or Nifty Trader, which provide intraday, minute-by-minute OI shift graphs. NiftyTrader +1
- **Rely on Price Action for Micro-moves:** Use 1-minute or 5-minute price action charts for your quick scalps, using OI only as a structural tool to define support/resistance zones rather than a micro-second trigger. Groww +2
- **Track Option Writers (Sellers):** Analyze OI changes to identify where option writers are building large positions (strikes with massive call/put writing act as strong ceilings or floors). If prices approach a heavy call-writing zone and OI drops on puts, look for a reversal. Groww +2

For a visual breakdown of how large option sellers use OI dynamics to spot support, resistance, and market trends throughout the day:

How to Use Open Interest for Intraday Option Trading

A&A Trading Blog
YouTube · 29 Jul 2025

If you want, let me know:

- Your **preferred broker/terminal**
- Which **specific strikes/indices** you usually trade (Nifty, Bank Nifty, etc.)

I can help you set up an optimal workspace that minimizes data lag for your intraday trades.

12 sites

Live NSE Option Chain: NIFTY, BANKNIFTY, Stocks - Sensibull
Option Chain. Home · Analyse · Positions · Watchlist. Login with your broker for real-time prices and trading. Login....
Sensibull

Nifty Open Interest Live — Strike-Wise OI Build-Up & Change
Auto-Refresh for Accurate Insights With the auto-refresh option, the Nifty OI chart...
NiftyTrader

Advanced Option Chain Analysis: Strategies, Indicators & Insights
Example 1: Normal trading day using OI (Option Selling) Here is a strategy in option...
Groww

Show all

Can I also AUGMENT , with ATM +/- 5 strike OI / VOLUME Change

3:27 pm

Yes, augmenting your intraday trading setup with the **At-The-Money (ATM) +/- 5 strikes Open Interest (OI) and Volume Change** is an excellent strategy. In fact, professional traders call this tracking the **"Active Zone"** or **"Battleground Strikes."**

Focusing on these 11 strikes (ATM, 5 In-The-Money, and 5 Out-Of-The-Money) eliminates market noise and highlights exactly where institutional money is moving.

Why the ATM +/- 5 Strike Strategy Works

- **Massive Liquidity Focus:** Over 80% of intraday volume and OI shifts happen within these 11 strikes. Far Out-Of-The-Money (OTM) strikes rarely impact immediate price direction.
- **Instant Sentiment Identification:** Tracking the cumulative OI change of these specific strikes tells you if option writers are aggressively selling calls (bearish) or puts (bullish) right at the current market price.
- **Early Breakout Detection:** A sudden spike in trading **Volume** in this zone, accompanied by a drop in **OI**, indicates short-covering. This is the primary driver behind explosive, fast-moving intraday rallies.

How to Implement This Setup

1. **Calculate Cumulative PCR (Put-Call Ratio):** Do not look at the PCR of the entire option chain. Calculate the PCR *only* for these 11 strikes. A rising PCR in this zone

If you are setting this up, let me know:

- Whether you want to trade **breakouts** or **reversals**
- If you need the exact **excel/python logic** to automate tracking this specific 11-strike zone

I can provide the precise steps or code to help you build this custom tracker!

One Q: the STRIKE wise OI DATA is coming FROM BROKER , who itself will get from EXCHANGE NSE and it means the ...

3:29 pm

Yes, you are exactly right. At its core, the raw Open Interest (OI) data for individual strikes originates from the exchange, meaning **all brokers are inherently bound by the NSE's data dissemination frequency.** kite.trade +1

However, there is a massive operational difference between how the public NSE India Website updates and how a broker's trading terminal processes that same data. Understanding this distinction is vital for an intraday trader.

The Mechanism: Free Web Lag vs. Paid Broker Feed

- **The NSE Website Lag (3 to 5 Minutes):** The free public NSE Option Chain undergoes heavy HTML rendering and server caching before it displays on your screen. This creates a staggered **3-to-5-minute structural delay**, which can stretch up to 6 minutes during peak market volatility. kite.trade +1
- **The Broker API Feed (1-Minute Intervals):** Commercial data vendors (like TrueData or Global Datafeeds) and brokers pay heavy subscription fees to pull raw binary snapshot files directly from NSE Data & Analytics servers. The exchange pushes these underlying OI files at a **strict 1-minute interval**. NSE India +2
- **The "Real-Time" Illusion:** When analytics platforms claim "real-time OI," they mean their systems parse and plot that raw 1-minute exchange data packet the exact millisecond it drops. You get the data on your chart a few minutes faster than a public web user, but **no broker has true tick-by-tick (second-by-second) Open Interest updates.** Global Datafeeds +1

Summary Comparison: Why Broker Data is Faster

Feature	Public NSE Website	Advanced Broker / Option Analytics Platform
Data Source	Delayed Web Snapshot	Raw Binary Exchange Feed via SFTP/API
Actual Interval	3 to 5 minutes lag	**Strict 1-minute intervals**
Visualization	Static table columns	Multi-strike dynamic continuous line charts
System Latency	High (web page refresh required)	Ultra-low (instant algorithmic calculation)

How Scalpers Handle the 1-Minute OI Lag

Because you cannot get second-by-second OI data, successful intraday options traders treat OI as a **structural filter rather than an execution trigger.**

They identify the heavy ATM +/- 5 strike build-up zones at the turn of every minute, but use **real-time Volume and 1-minute Candlestick Price Action** to time their exact entries and exits. Volume updates with every single transaction (tick-by-tick), allowing you to spot an institutional order block long before it registers in the next minute's OI packet. kite.trade +2

If you would like to explore this further, let me know:

- Would you like to see how to use **Tick-by-Tick Order Flow** or **Volume Profiles** to bypass the 1-minute OI lag entirely?
- Do you want a breakdown of how to identify **Short Covering rallies** using the mismatch between instant Volume and the 1-minute OI update?

9 sites

Real Time Data - NSE India
Paid Real time data * Paid Real Time Data. * Paid Analytical Products. * Paid Non-Display Data. * Paid Historical Data. *...
NSE India

Global Datafeeds - NSE,BSE & MCX Authorized Data Vendor
9) Low-Latency Real-Time Data by NSE Data Vendor. During trading hours, the real...
Global Datafeeds

Live Nifty options data? - Kite Connect developer forum
Look,first of all,know this that KiteConnect doesn't provide any data different than kite web/app. It's the same data but...
kite.trade

Show all

You have hit the nail on the head. Because Volume and Option Prices update **tick-by-tick (sub-second)**, combining them is the absolute best way to catch a shifting bias long before the official 1-minute OI data packet drops.

When you track the **Volume and Price change of the ATM +/- 5 strikes**, you are essentially building a real-time proxy for institutional momentum.

The Real-Time Tracking Framework

Instead of waiting for OI, you monitor the relationship between **Traded Volume** and **Premium Price Movement** across your 11-strike zone. This gives you four distinct intraday signals:

- **Bullish Dominance (Long Buildup Proxy):** Call Option (CE) Volume spikes + CE Prices rise rapidly. At the same time, Put Option (PE) Prices decay. This means buyers are aggressively chasing calls.
- **Bearish Dominance (Short Buildup Proxy):** Put Option (PE) Volume spikes + PE Prices rise rapidly. At the same time, CE Prices collapse. This means buyers are aggressively loading puts.
- **The Short-Covering Rocket (The Scalper's Dream):** CE Price shoots up vertically while CE Volume is extraordinarily high. This tells you that call sellers are panicking and buying back their positions at any available market price.
- **The Long Unwinding Trap:** Prices drop sharply on high volume. This shows buyers are dumping their options to cut losses.

---

3 Metrics to Build Your Custom Real-Time Dashboard

To turn this into an actionable trading setup, you can calculate three live metrics using the sum of your ATM +/- 5 strikes:

1. The Real-Time Volume PCR (Put-Call Ratio)

$$\text{Volume PCR} = \frac{\text{Total Traded Volume of PE (+/- 5 Strikes)}}{\text{Total Traded Volume of CE (+/- 5 Strikes)}}$$

- **How to use it:** Traditional OI PCR is slow. **Volume PCR is instantaneous.** If the Nifty is consolidating but the Volume PCR suddenly jumps from 0.8 to 1.4, institutional money is silently flooding into puts. A downward price break is imminent.

2. Momentum Velocity Mismatch

Track the speed of price movement. If the underlying index (e.g., Nifty Spot) moves up by 10 points, but the cumulative price of your selected CE strikes shoots up by an absolute premium value disproportionately faster than normal delta behavior, it indicates massive aggressive buying.

3. Real-Time Option Order Flow (Tape Reading)

Look at the **traded volume at the Ask price vs. Bid price** within those 11 strikes.

- If massive volume blocks are hitting the **Ask (Seller's price)** on CE, traders are aggressively buying calls.
- If massive volume blocks are hitting the **Bid (Buyer's price)** on CE, institutional sellers are heavily shorting calls.

Summary: Your Intraday Execution Loop

```
[OI Data (Every 1 Min)] ──> Establishes the Day's Macro Bias / Major Support & Re
       │
       ▼
[ATM +/- 5 Volume & Price (Live)] ──> Spots Micro-Shifts & Institutional Traps
       │
       ▼
[1-Min Price Action Chart] ──> Absolute Timing for Triggering the Trade
```

If you want to build or automate this setup, let me know:

- Do you use a terminal that allows **Excel API integration** (like Kite Connect, Angel One, or Dhan)?
- Would you like the mathematical logic/formulas to set up a **Live Volume Scanner** in Excel or Python?

Because the raw underlying exchange data packet drops at a **1-minute interval** via commercial API feeds, and the public web updates every **3 to 5 minutes**, the macro loop depends entirely on which platform you are looking at:

```
[OI Data (1 to 5 Min Lag)] ──> Establishes the Day's Macro Bias / Structural Wall
       │
       ▼
[ATM +/- 5 Volume & Price (Instant)] ──> Spots Real-Time Shifts & Institutional T
       │
       ▼
[1-Min Price Action Chart] ──> Absolute Timing for Triggering the Entry/Exit
```

Why the Distinction Matters for Your Setup

Whether it is 1 minute (via paid broker data feeds) or 5 minutes (via the free NSE website), **it is still too slow for an execution trigger** when premium prices are moving second-by-second.

That is why your idea of using the **ATM +/- 5 Strike Volume and Price Change** is the perfect solution. It bridges that data lag beautifully. You let the 3-to-5-minute OI tell you where the "big structural walls" are, but you use the live Volume and Price action to time your actual entries before the next OI packet even registers.

If you are planning to build this out, let me know:

- Would you like the formula logic to calculate the **Real-Time Volume PCR** in Excel?
- Do you want to see how to identify a **Short Covering rally** using just the instant premium price spikes and volume?

I have option data in format so using our logic we can focus on nearest +-5 strikes and so i compressed (eliminated data...

4:49 pm

This is an excellent historical dataset structure for backtesting and logic optimization. Since you have successfully compressed your database to only capture the 11 key strikes (ATM +/- 5), your calculation overhead will drop drastically, making your analytics loop incredibly efficient.

Looking closely at your dataset sample, there is a **very specific data behavior** you need to account for when programming your logic.

⚠️ Critical Discovery in Your Data Sample

Look at the Open Interest (OI) column across the 4 minutes of your sample for the **23500 CE** strike:

- **09:15:00 → OI = 20,605**
- **09:16:00 → OI = 650**
- **09:17:00 → OI = 0**
- **09:18:00 → OI = 0**

This structural pattern is a textbook example of exactly what we discussed. Because the **exchange does not push OI every single minute**, data providers and broker databases handle the non-update minutes in one of two ways depending on how your database was populated:

1. **The Clear-to-Zero Method:** The provider populates real-time tick volume and price continuously, but clears the OI field to `0` or leaves it blank on minutes when the exchange has not broadcasted a fresh snapshot packet.
2. **Illiquid/Far Expiry Drop:** Alternatively, if this represents an un-updated snapshot, it creates a massive false drop in your algorithms.

How to Prepare Your Logic for This Data structure

To make your ATM +/- 5 tracking logic robust against these 3-to-5-minute data gaps, apply the following structural steps to your data parsing code (Python, SQL, or Excel):

- **Forward-Fill (FFILL) the OI Column:** Before calculating any metric, fill the `0` or missing OI values with the last known non-zero value. At `09:17:00`, your code should treat the OI as `650`, not `0`, so your net intraday OI change calculations do not break.

calculation loop must group rows by `Timestamp` to aggregate the metrics.

The Aggregation Formula for Your Tracking Engine

For every specific `Timestamp` block in your database, your code should map and calculate this structural framework:

```python
# Conceptual Python/Pandas logic for your database loop
# 1. Filter data for the specific timestamp
# 2. Separate into CE and PE dataframes within your filtered ATM +/- 5 rows

total_ce_volume = ce_df['Volume'].sum()
total_pe_volume = pe_df['Volume'].sum()

# Instantaneous Volume PCR
volume_pcr = total_pe_volume / total_ce_volume if total_ce_volume > 0 else 0

# Weighted Average Premium Price tracking
ce_weighted_close = (ce_df['Close'] * ce_df['Volume']).sum() / total_ce_volume
```

Use code with caution.

If you'd like, let me know:

- Is your processing environment built in **Python (Pandas)**, **SQL**, or **Excel**?
- Is the underlying **Nifty Spot index price** stored in a separate table, or do you need code to dynamically derive the **ATM strike** directly from these option premiums?

I can provide the exact code block to automate the calculation of your **Real-Time Volume PCR** and **Price Velocity** shifts!

Show Code <>

AI responses may include mistakes. For financial advice, consult a professional. Learn more

i have created weekly databased to avoid very big DB as other data we are not using except SPOT may be needed for...

4:51 pm

Using SQLite with weekly database files (around 40 MB each) is a highly efficient architecture. It keeps your data modular, prevents memory bloating, and makes backtesting individual market phases incredibly fast.

Since you are using **Python** and **SQLite**, we can build a highly optimized script that applies our logic. This script handles the 3–5 minute OI gaps using forward-fill, filters the dynamic **ATM ± 5 strikes** using a companion Nifty Spot table, and aggregates the real-time Volume and Price momentum.

Database Assumptions

To run this logic, we assume your SQLite database contains two tables for the week:

1. `options_data`: Containing the columns you provided (`Date`, `Timestamp`, `Volume`, `OI`, `Close`, `Ticker`, `option_type`, `strike` *derived from ticker or your dataset*).
2. `spot_data`: Containing `Timestamp` and `Close` (Nifty Spot price) to dynamically identify the ATM strike at any given minute.

The Python/Pandas Integration Script

Here is the production-ready script to extract, clean, and calculate your real-time proxies:

```python
import sqlite3
import pandas as pd
import numpy as np

# 1. Connect to your weekly SQLite database
conn = sqlite3.connect("nifty_weekly_2026_04_21.db")

# 2. Load Spot and Options data into Pandas DataFrames
spot_query = "SELECT Timestamp, Close as Spot_Close FROM spot_data;"
```

```python
# 3. Merge Spot Price onto Options DataFrame to calculate ATM on every row
df = pd.merge(df_options, df_spot, on='Timestamp', how='inner')

# 4. Handle the Exchange OI Lag (Fixing the 0/missing data issue)
# Replace 0 with NaN so we can forward-fill within each unique contract Ticker
df['OI'] = df['OI'].replace(0, np.nan)
df = df.sort_values(by=['Ticker', 'Timestamp'])
df['OI'] = df.groupby('Ticker')['OI'].ffill().fillna(0)

# 5. Dynamically Filter the "Battleground Strikes" (ATM +/- 5)
# Nifty strike intervals are 50 points. 5 strikes out = 250 points.
df['Strike_Diff'] = (df['strike'] - df['Spot_Close']).abs()
df_filtered = df[df['Strike_Diff'] <= 250].copy()

# 6. Group by Timestamp to create your Real-Time Indicators
aggregated_data = []

for timestamp, group in df_filtered.groupby('Timestamp'):
    # Separate CE and PE for this specific minute
    ce_group = group[group['option_type'] == 'CE']
    pe_group = group[group['option_type'] == 'PE']

    # Calculate Volumes
    total_ce_vol = ce_group['Volume'].sum()
    total_pe_vol = pe_group['Volume'].sum()

    # Calculate Real-Time Volume PCR (Avoid division by zero)
    volume_pcr = total_pe_vol / total_ce_vol if total_ce_vol > 0 else 0

    # Calculate Volume-Weighted Average Premium Price for Momentum Velocity
    ce_vwap_price = (ce_group['Close'] * ce_group['Volume']).sum() / total_ce_vo
    pe_vwap_price = (pe_group['Close'] * pe_group['Volume']).sum() / total_pe_vo

    # Calculate Aggregated Cleaned OI (for structural bias)
    total_ce_oi = ce_group['OI'].sum()
    total_pe_oi = pe_group['OI'].sum()
    oi_pcr = total_pe_oi / total_ce_oi if total_ce_oi > 0 else 0

    aggregated_data.append({
        'Timestamp': timestamp,
        'Spot_Price': group['Spot_Close'].iloc[0],
        'CE_Volume': total_ce_vol,
        'PE_Volume': total_pe_vol,
        'Volume_PCR': volume_pcr,
        'CE_Weighted_Price': ce_vwap_price,
        'PE_Weighted_Price': pe_vwap_price,
        'OI_PCR_Structural': oi_pcr
    })

# Convert to final analytics DataFrame
df_signals = pd.DataFrame(aggregated_data)
print(df_signals.head())
```

Use code with caution.

Performance & Storage Tip for SQLite (Keep it at 40MB)

To ensure this query runs in seconds rather than minutes across your 40 MB files, you **must create database indexes**. Execute these two SQL commands in your SQLite database manager once when creating the weekly file:

sql

```sql
CREATE INDEX idx_options_timestamp ON options_data (Timestamp);
CREATE INDEX idx_options_ticker_time ON options_data (Ticker, Timestamp);
```

Use code with caution.

Without these indexes, Python will perform a slow, brute-force "table scan" every single time you query a week. With them, SQLite will fetch the dynamic 11 strikes instantly.

If you are ready to use this, let me know:

- Do you want to add the code logic to detect **Short Covering Signals** (when Price Velocity spikes but forward-filled OI drops)?
- How do you plan to handle **Nifty Expiry days** when the ATM strikes change rapidly within minutes?